

Programming Lab 03: Saving Our Session

V. Markos
Mediterranean College

Fall, 2024 – 2025, Week 07

Abstract

What is this? Probably an ongoing lab for this course, where you will create simple game in Python. Do not expect spectacular graphics and all that stuff, since this course is not about that, but expect to encounter and attack several data structures and algorithm-related problems as we go into more sophisticated designs for our game. Stay tuned for more...

1 Last Time Recap

Last time we extended our game with the following features (at least):

- The ability to move around the grid and keep track of that movement, indicated on the game grid by a ..
- Relics which are initially invisible on the game grid and can be collected by the player, granting them some points.
- Graceful handling of a simple game-over condition, i.e., when all relics have been collected by the player.
- Optionally, you might have also implemented more complex relic semantics, i.e., have more than one types of relics. In that case, you must have also properly handled the game-over logic (“how” is up to you).
- Optionally, you might have also implemented a “drop relic” functionality, where each player is allowed to ignore a relic by not collecting it the time they encounter it.

2 Save Me!

This is not a cry for help¹, but a realistic demand from any sufficiently complex game (and our game aims to be one). What if a player gets bored in the middle

¹Well, it might also be one, but do not care about that at the moment...

of a game session or something emergent occurs? Can they pause their game and resume later?

Of course they can, in the sense of leaving their machine open and hanging at that point but is there not any better way to do so? One option, as you might have guessed is to somehow save the game's state so far and save it onto the local disk. So, here comes your first task:

Task 1: Exporting as Text

You should find a way to export your game's state as text. This should include, at least, the following:

- The grid's state, i.e., which cells remain covered by the FoW, and, consequently, which have already been explored.
- The player's location on the grid.
- The score of the player so far (according to whichever way you implement player scoring).
- The relics positions (at least of those that are yet to be explored).

You can also any further information you might need to facilitate exporting and (maybe) re-importing your game. Also, to make your implementation functional, you have to somehow allow the player decide when to save / export a game — e.g., by providing a special character through the terminal. The output file should have a `.txt` extension.

Now that we have a valid format for our game in `.txt`, we would like to be able to import a game we have previously exported. This actually means to load into our computer's memory (RAM) all relevant information included in the exported text file. So...

Task 2: Loading From a File

Utilising the export text format you have devised for Task 1, implement any required functionality that will allow you to load all relevant game information from the exported text file. This means that, after loading a previously exported game, the user should get the impression that they are resuming exactly from where they were left when they last saved their session!

3 Help!

We have already implemented some interesting stuff regarding our game: movement, collectible relics, import and export utilities... But, how does the player

know how to handle all these? We should implement some sort of a help page, which the player should be able to trigger to present some stuff on screen.

Task 3: Game Help

Write a help file stored as `help.txt` in an appropriate location in your project. In this file you should explain all functionality offered by the game and how the player can access it. For instance, you should explain things like:

- How the player moves.
- What is the objective of the game.
- How a player can export / import their game session.
- How they can access this help page.

Then display that file using an appropriate user prompt. For instance, you could display the help file whenever the user enter `--help` as their next move. Or you could read this as a parameter from the terminal (you might need Python's `os` and / or `sys` libraries for that).

4 Further Considerations

While the text import functionality we have implemented today might work as expected, text files are not, in general, always the best option to work with. For instance, JSON files might be a better way to store information in a structured format. Also, JSON files can be (most of the times) directly loaded as Python dictionaries using Python's `json` module.

Task 4: Using JSON

Reimplement all functionalities you have so far for Tasks 1 and 2 using JSON files. Take care of the file formatting, its structure and how you can take advantage of them to facilitate the import / export processes.